

# PHASE 5 — Tasks, Projects & Client Management

## PRSECURITY HRMS

### IMPORTANT RULES

Output every file 100% completely. Never use '...' or 'rest remains same'.

After each file write "--- FILE COMPLETE ---" then start the next file immediately.

If about to hit output limit, finish current file and write "--- PAUSED: Type CONTINUE ---"

This is Phase 5 of 8. Phases 1-4 done: Auth, Employees, Dashboards, Attendance, Leave, Payroll.

DO NOT rebuild previous files. Only add NEW files or EXTEND where specified.

### CONTEXT

Project: PRSECURITY HRMS | Stack: React+Vite+TS+Tailwind+shadcn/ui |

Node+Express+Prisma+PostgreSQL Roles: ADMIN, HR, EMPLOYEE, INTERN | Currency: ₹

### STEP 1 — Update Prisma Schema: Task, Project, Client Models



prisma

```
model Client {
  id          String  @id @default(uuid())
  name        String
  email       String?
  phone       String?
  address     String?
  industry    String?
  contactPersonName String?
  contactPersonEmail String?
  contactPersonPhone String?
  gstin       String?
  website     String?
  status      ClientStatus @default(ACTIVE)
  createdBy   String
  creator     User  @relation(fields: [createdBy], references: [id])
  projects    Project[]
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
}
```

```
enum ClientStatus { ACTIVE INACTIVE }
```

```
model Project {
  id      String  @id @default(uuid())
  name    String
  description String?
  clientId String?
  client  Client?  @relation(fields: [clientId], references: [id])
  status  ProjectStatus @default(PLANNING)
  startDate DateTime?
  endDate DateTime?
  budget  Float?
  priority Priority  @default(MEDIUM)
  createdBy String
  creator  User  @relation(fields: [createdBy], references: [id])
  members  ProjectMember[]
  tasks    Task[]
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
}
```

```
enum ProjectStatus { PLANNING ACTIVE ON_HOLD COMPLETED CANCELLED }
```

```
enum Priority { LOW MEDIUM HIGH CRITICAL }
```

```

model ProjectMember {
  id      String @id @default(uuid())
  projectId String
  userId  String
  role    String @default("MEMBER")
  joinedAt DateTime @default(now())
  project Project @relation(fields: [projectId], references: [id])
  user    User   @relation(fields: [userId], references: [id])
  @@unique([projectId, userId])
}

model Task {
  id      String @id @default(uuid())
  title    String
  description String?
  projectId String?
  project  Project? @relation(fields: [projectId], references: [id])
  assignedTo String
  assignedBy String
  assignee  User   @relation("TaskAssignee", fields: [assignedTo], references: [id])
  assigner  User   @relation("TaskAssigner", fields: [assignedBy], references: [id])
  priority  Priority @default(MEDIUM)
  status    TaskStatus @default(TODO)
  dueDate   DateTime?
  completedAt DateTime?
  estimatedHours Float?
  actualHours Float?
  comments  TaskComment[]
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
}

enum TaskStatus { TODO IN_PROGRESS IN_REVIEW DONE CANCELLED }

model TaskComment {
  id      String @id @default(uuid())
  taskId  String
  userId  String
  content String
  task    Task   @relation(fields: [taskId], references: [id])
  user    User   @relation(fields: [userId], references: [id])
  createdAt DateTime @default(now())
}

```

Run: `npm run prisma migrate dev --name add_tasks_projects_clients`

## STEP 2 — Client Backend

### FILE: backend/src/services/client.service.ts

`getAllClients(query)` — paginated, search by name, filter by status  
`createClient(data, createdBy)` — create client  
`getClient(id)` — client detail with projects list  
`updateClient(id, data)` — update  
`deactivateClient(id)` — soft delete (status=INACTIVE)  
`getClientStats()` — total active, total inactive, total projects across clients

### FILE: backend/src/controllers/client.controller.ts

### FILE: backend/src/routes/client.routes.ts

```
□  
  
GET   /api/clients      → getAllClients (HR/Admin)  
POST  /api/clients      → createClient (HR/Admin)  
GET   /api/clients/stats → getClientStats (HR/Admin)  
GET   /api/clients/:id  → getClient (HR/Admin)  
PUT   /api/clients/:id  → updateClient (HR/Admin)  
DELETE /api/clients/:id  → deactivateClient (Admin)
```

## STEP 3 — Project Backend

### FILE: backend/src/services/project.service.ts

`getAllProjects(userId, role, query)` — HR/Admin see all; Employee/Intern see only projects they're a member of  
`createProject(data, createdBy)` — create + optionally add initial members  
`getProject(id, userId, role)` — detail with members list, client info, task summary (counts by status)  
`updateProject(id, data)` — update (HR/Admin or project creator)  
`archiveProject(id)` — set status=CANCELLED (Admin)  
`addMember(projectId, userId, role)` — add member (HR/Admin)  
`removeMember(projectId, userId)` — remove member (HR/Admin)  
`getProjectTasks(projectId)` — all tasks for project  
`getProjectStats()` — count by status

### FILE: backend/src/controllers/project.controller.ts

### FILE: backend/src/routes/project.routes.ts

```
□
```

GET /api/projects → getAllProjects (authenticated)  
 POST /api/projects → createProject (HR/Admin)  
 GET /api/projects/stats → getProjectStats (HR/Admin)  
 GET /api/projects/:id → getProject (authenticated)  
 PUT /api/projects/:id → updateProject (HR/Admin)  
 DELETE /api/projects/:id → archiveProject (Admin)  
 POST /api/projects/:id/members → addMember (HR/Admin)  
 DELETE /api/projects/:id/members/:userId → removeMember (HR/Admin)  
 GET /api/projects/:id/tasks → getProjectTasks (authenticated)

## STEP 4 — Task Backend

**FILE: backend/src/services/task.service.ts**

`getAllTasks(userId, role, query):`

Employee/Intern: only tasks assigned TO them

HR/Admin: all tasks

Filters: status, priority, projectId, assignedTo, dueDateRange, page, limit

`createTask(data, createdBy):`

Create task

Send notification to assignee: "New task assigned: {title}"

`getTask(id, userId, role):`

Detail with comments, assignee info, project info

Employee can only see if assigned to them

`updateTask(id, data, userId, role):`

HR/Admin can update anything

Assignee can update: status, actualHours, completedAt (auto-set when status=DONE)

`updateTaskStatus(id, status, userId):`

Quick status update

If status=DONE: set completedAt=now

Notify task creator of status change

`deleteTask(id, userId, role):`

Admin/HR or creator can delete

`addComment(taskId, userId, content):`

Create TaskComment

Notify task assignee and creator (if different from commenter)

`getComments(taskId):`

All comments with user info

`getTaskStats(userId?, role?):`

Count by status (TODO, IN\_PROGRESS, IN\_REVIEW, DONE)

If userId provided: for that user. Else: all tasks.

**FILE: backend/src/controllers/task.controller.ts**

**FILE: backend/src/routes/task.routes.ts**



GET /api/tasks → getAllTasks (authenticated)  
POST /api/tasks → createTask (authenticated)  
GET /api/tasks/stats → getTaskStats (authenticated)  
GET /api/tasks/:id → getTask (authenticated)  
PUT /api/tasks/:id → updateTask (authenticated)  
DELETE /api/tasks/:id → deleteTask (authenticated)  
PUT /api/tasks/:id/status → updateTaskStatus (authenticated)  
POST /api/tasks/:id/comments → addComment (authenticated)  
GET /api/tasks/:id/comments → getComments (authenticated)

## STEP 5 — Client Frontend

**FILE: frontend/src/api/client.api.ts**

**FILE: frontend/src/pages/clients/ClientsPage.tsx**

PageHeader: "Clients" + "Add Client" button

StatCards: Total Active Clients, Total Projects

Card grid of clients (not table — cards look better for clients):

Each card: Company name, industry badge, contact person, email, phone, active projects count, status badge

Actions: View, Edit, Deactivate

Search by name

Click card → /clients/:id

**FILE: frontend/src/pages/clients/ClientDetailPage.tsx**

Back button

Client info section: all fields displayed

"Edit" button → opens edit modal

Projects section: list of projects for this client with status badge, progress bar, team avatars

**FILE: frontend/src/components/clients/ClientFormModal.tsx**

Create/Edit modal with all client fields: company name, email, phone, address, industry, GSTIN, website, contact person name/email/phone.

## STEP 6 — Project Frontend

**FILE: frontend/src/api/project.api.ts**

**FILE: frontend/src/components/projects/ProjectCard.tsx**

Card component showing:

Project name + priority badge

Client name (if linked)

Status badge

Team member avatars (max 4 shown, +N for rest)

Progress bar: % tasks completed

Start date → End date

Click → /projects/:id

## **FILE: frontend/src/pages/projects/ProjectsPage.tsx**

PageHeader: "Projects" + "New Project" button (HR/Admin)

Toggle: Card view / Table view

Card view: grid of ProjectCards

Table view: DataTable with columns: Name, Client, Status, Priority, Team Size, Progress, Start, End, Actions

Filters: Status, Priority

## **FILE: frontend/src/pages/projects/ProjectDetailPage.tsx**

Tabs:

Tab 1 — Overview:

Project name, description, client, dates, budget (₹), priority badge, status badge

Edit button (HR/Admin)

Progress stats: tasks by status as small bars

Tab 2 — Team:

Member list: avatar, name, role badge, joining date

"Add Member" button → dropdown to select employee + role

Remove member button (HR/Admin)

Tab 3 — Tasks:

Embedded mini Kanban board filtered to this project

"Add Task" button

Tab 4 — Timeline:

Simple horizontal Gantt view using Recharts or plain CSS bars

Each task as a row: task name + coloured bar spanning start-end dates

## **FILE: frontend/src/components/projects/ProjectFormModal.tsx**

Create/Edit project modal:

Name, Description, Client dropdown, Start Date, End Date, Priority, Budget (₹)

Initial team members multi-select

## **STEP 7 — Task Frontend**

### **FILE: frontend/src/api/task.api.ts**

### **FILE: frontend/src/components/tasks/KanbanBoard.tsx**

Full Kanban board with 4 columns: To Do | In Progress | In Review | Done

Each card shows:

Title (truncated)

Priority badge (colour coded: LOW=grey, MEDIUM=blue, HIGH=orange, CRITICAL=red)

Assignee avatar

Project tag (if linked)

Due date (red if overdue)

Comment count

Clicking a card opens TaskDetailDrawer. Drag-and-drop between columns using @hello-pangea/dnd (install this package). On drop: call updateTaskStatus API. "+ Add Task" button at top of each column opens CreateTaskModal pre-filled with that status.

### **FILE: frontend/src/components/tasks/TaskDetailDrawer.tsx**

Right-side drawer (sheet) showing full task detail:

Title + priority badge + status dropdown (assignee can change)

Description

Assignee info, Assigned by info

Project link

Due date, Estimated hours, Actual hours input

Comments thread: list of comments with avatar + name + time, Add comment textarea + submit

Activity log (created/updated events)

Edit button (HR/Admin or creator)

Delete button (HR/Admin or creator)

**FILE: frontend/src/components/tasks/CreateTaskModal.tsx**

Form: Title\*, Description, Project dropdown, Assign To\* (employee dropdown), Priority, Due Date, Estimated Hours, initial Status.

**FILE: frontend/src/pages/tasks/TasksPage.tsx**

PageHeader: "Tasks" + "New Task" button

Toggle: Kanban / List view

Kanban: KanbanBoard component

List view: DataTable with columns: Title, Assignee, Project, Priority, Status, Due Date, Actions

Filters (list view): Status, Priority, Assignee, Project

My Tasks / All Tasks toggle (HR/Admin can see all; Employee sees own by default)

## STEP 8 — Connect to Dashboards

**Update backend/src/services/dashboard.service.ts**

Admin: fill in `activeProjects` count

Employee: fill in `taskStats` with real counts from tasks table

**Update frontend/src/pages/dashboard/AdminDashboardPage.tsx**

Connect activeProjects StatCard

**Update frontend/src/pages/dashboard/EmployeeDashboardPage.tsx**

Show real task stats (To Do / In Progress / Done count cards)

Show "My Tasks Due This Week" list (3-5 tasks)

## AFTER ALL FILES ARE COMPLETE

Write this exactly: "  PHASE 5 COMPLETE — Task Management (Kanban board, drag-and-drop, comments), Project Management (team, timeline, progress), and Client Management are ready. Proceed to Phase 6 for Recruitment & Performance Management."